

DataScience -- Network Intrusion Traffic Analysis

Intrusion Detection Systems (IDS) rely on labelled network traffic to learn the difference between benign and malicious flows. The challenge is that no single public dataset captures the full diversity of modern attacks: CICIDS2017 focuses on simulated enterprise traffic with modern attacks (DDoS, brute-force, web attacks, infiltration), UNSW-NB15 mixes synthetic exploits and fuzzers, and NSL-KDD remains the historical reference for probe / R2L / U2R activity. This lab works on a **unified IDS corpus** of ~133,000 network flow records merged from these three sources and re-labelled into a common 16-class taxonomy (BENIGN, DOS, DDOS, PROBE, EXPLOIT, FUZZER, R2L, U2R, BRUTE_FORCE, WEB_ATTACK, INFILTRATION, BOT, BACKDOOR, ANALYSIS, SHELLCODE, WORM).

The dataset `data.csv` contains one row per flow, with shared numeric features (duration, `src_bytes`, `dst_bytes`, `packet_count`, `ports`, `protocol`), the unified `label`, the original `label_detail`, the source identifier, and a JSON-encoded `extra_features` field carrying source-specific signals (TCP flag counts, flow byte/packet rates, behavioural indicators). The dataset documentation file `SUMMARY.md` is provided as a reference.

The goal of this lab is to practice **intrusion detection** on a realistic multi-class corpus, ending with a personal end-to-end ML project of the student's own design. Scikit-learn is the default toolkit.

Part 1 -- Dataset Exploration

1. Please load the dataset `data.csv` using pandas or an equivalent tool. Document the following:
 - Number of rows and columns
 - Data types of each column
 - Number of missing values per column (many columns are nullable because the three sources expose different feature sets -- e.g. `src_port` is CICIDS-only)
 - Distribution of the `source` field (how many flows come from CICIDS2017, UNSW-NB15, NSL-KDD?)
 - Distribution of the `label` field: count of BENIGN flows vs. each attack class. Produce a horizontal bar chart sorted by frequency.

How many distinct attack classes are present in the unified taxonomy? Is the dataset balanced?

2. Please profile the numerical fields `duration`, `src_bytes`, `dst_bytes`, `packet_count` and compare them across sources:
 - Compute descriptive statistics (mean, median, min, max, standard deviation) for each numeric field.
 - Produce histograms on a **logarithmic scale** (`log1p` transform or `plt.yscale("log")`) for each of them. Comment briefly on the heavy-tailed shape.
 - Produce **side-by-side boxplots** (one panel per feature, three boxes per panel -- one per source) for the four numerical features.

Do the three sources cover the same numeric ranges? Are there systematic differences (e.g. NSL-KDD durations vs. CICIDS durations)? What does this imply for a model trained on one source and tested on another?

Part 2 -- Protocol, Ports, and Source/Label Structure

An **Intrusion Detection System (IDS)** is a tool that monitors network traffic and flags activity that looks malicious. It usually works at the level of **network flows** -- aggregated summaries of all packets exchanged between two endpoints (source IP/port, destination IP/port, protocol) over a short time window, with statistics such as duration, byte counts and packet counts. In a labelled IDS dataset, every flow is tagged either as **BENIGN** (normal traffic) or as one of several **attack** classes (denial-of-service, port scan, brute-force login, exploit attempt, etc.). The three sources in this corpus -- **CICIDS2017**, **UNSW-NB15** and **NSL-KDD** -- are widely used IDS benchmarks that capture flows from simulated enterprise networks under both normal use and a curated set of attack scenarios, and then label every flow with the activity that generated it.

3. Please analyze the `protocol` and `dst_port` fields:
 - Global distribution of `protocol` (`tcp`, `udp`, `icmp`, other). Produce a bar chart.
 - Conditional on `protocol`, distribution of `label`. Produce a stacked or grouped bar chart showing, for each protocol, the proportion of BENIGN vs. each attack class. Which protocols carry which attacks (e.g. is ICMP dominated by PROBE, is UDP dominated by DOS / DDOS)?
 - For `dst_port` (where present -- CICIDS and UNSW-NB15), compute the **top 20 destination ports by frequency** and cross-check them against well-known service ports (22, 23, 80, 443, 445, 3389, 8080).

4. Please produce a **cross-tabulation** source × label (rows = source, columns = label) and visualise it as a heatmap.
- Which attack classes are missing from each source? (For instance, NSL-KDD has no WEB_ATTACK, CICIDS has no FUZZER / U2R, UNSW-NB15 has its own taxonomy that maps differently.)
 - Discuss the **label-set heterogeneity**: what are the consequences when training a multi-class classifier on flows that physically cannot contain certain classes from certain sources?

Part 3 -- Feature Engineering & Class Imbalance

5. Please construct the following **engineered features** on top of the raw columns:
- `bytes_ratio = src_bytes / (dst_bytes + 1)` -- captures asymmetry (scans send much more than they receive).
 - `packet_rate = packet_count / max(duration, 0.001)` -- packets per second.
 - `is_well_known_port = (dst_port < 1024)` -- boolean flag.
 - `log_src_bytes = log1p(src_bytes)` and `log_dst_bytes = log1p(dst_bytes)` -- stabilise heavy tails.
 - `log_duration = log1p(duration)`.

For at least two of the engineered features, plot the **distribution per label** (boxplot or violin plot, x = label, y = feature, log scale where appropriate). Comment on which features look discriminative (e.g. does `packet_rate` separate DOS from BENIGN? does `bytes_ratio` mark scans?).

6. Please quantify and address the **class imbalance**:
- Compute the proportion of each class. Identify the majority class and the rarest classes. What is the imbalance ratio between the most and least frequent class?
 - Use `class_weight='balanced'` (in scikit-learn) when you train your models in Part 4. Briefly explain in 2-3 sentences why class-weighting is a reasonable default here (no new synthetic samples, simple to apply, transparent).
7. Please write **2 simple rule-based detection rules** inspired by classical IDS heuristics. For example:
- `packet_rate > 10000 AND duration < 1` □ likely **DOS** / flood.
 - `bytes_ratio > 100 AND packet_count < 5` □ likely **PROBE** (scan).
 - `dst_port == 22 AND src_bytes < 200 AND packet_count > 20` □ likely **BRUTE_FORCE** (SSH).
 - `protocol == "icmp" AND packet_count > 50` □ likely **PROBE** (ping sweep).

For each rule, compute the **accuracy** of the rule when used as a binary classifier (predict the target class if the rule matches, else "other") on the full dataset. Are simple thresholds enough? Where do they fail?

Part 4 -- Modeling

8. **Binary intrusion detection.** Please build a binary classifier BENIGN vs. attack (any non-BENIGN label collapses to attack):
- Select the **common numerical features** present across all three sources: `duration`, `src_bytes`, `dst_bytes`, `packet_count`, plus your engineered features from exercise 5. Encode `protocol` with one-hot or ordinal encoding.
 - Split: 70% train / 15% validation / 15% test, **stratified by label**, with `random_state=42`.
 - Train **Random Forest** (`n_estimators=200`, `class_weight='balanced'`) and one **Gradient Boosting** model (`HistGradientBoostingClassifier`).
 - Report on the test set: accuracy, precision, recall, F1, ROC-AUC, and a confusion matrix.
 - Plot **feature importance** for the Random Forest. Which features dominate -- the engineered ones or the raw ones?
9. **Multi-class attack classification.** Please extend to the full taxonomy:
- Keep either the full 16 classes or the **top 8 most populated classes** (collapse the rest into OTHER). State your choice.
 - Train one Random Forest (with `class_weight='balanced'`) on the same features as exercise 8.
 - Report **macro-F1**, **weighted-F1**, and **per-class precision / recall** in a table.

- Produce a **confusion matrix** (normalised by row). Which classes are confused with which? Comment on systematic confusions (e.g. DOS vs. DDOS, PROBE vs. ANALYSIS). Is the model better at detecting frequent classes or rare ones?

Part 5 -- Personal ML Project

Now that you have explored, profiled and modelled the dataset, please design and execute a small end-to-end machine-learning project of your own choice on this same dataset.

1. **Define the task.** State one concrete problem this dataset can answer. Phrase it as a supervised, unsupervised, or semi-supervised learning task. Justify why this is a meaningful question, not a toy.
2. **Pick the model family.** Choose a model appropriate to the data and the task. Explain in 3-5 sentences why this family fits.
3. **Train and evaluate end-to-end.** Produce a clean pipeline: splits, pre-processing, tuning, metrics. Include at least one comparison baseline.
4. **Describe two or three concrete use cases.** Name the stakeholder, the decision the model supports, and the false-positive vs false-negative cost.
5. **Limitations and ethics.** Three honest limitations and one ethical/legal consideration.

© Angelo Consoli, SUPSI DTI Viganello